

1. What is an Ontology?

Definition: An ontology is a structured model of knowledge that defines:

- **Concepts (classes):** e.g., Customer, Supplier, Invoice, Shipment.
- **Relationships (properties):** e.g., Customer places Order, Supplier delivers Product.
- **Rules/constraints (axioms):** e.g., “Every invoice must have exactly one issuer.”

👉 Think of it as a map of your business world that both humans and machines understand.

2. Steps in Ontology Extraction

Ontology extraction = turning unstructured/messy data → structured ontology.

The pipeline has 8 steps. Below I’ll define the algorithms/terms you’ll see in each.

Step 1. Term Collection

What: Gather candidate terms from text, column names, schemas.

Techniques:

- **Tokenization** → split text into words.
 - Example: "Customer placed an order" → [Customer, placed, an, order].
- **Stopword removal** → remove filler words like “the,” “and.”
- **Stemming/Lemmatization** → reduce words to root.
 - Orders, ordering → order.

Step 2. Candidate Concept Extraction

What: Identify likely concepts (usually nouns or noun phrases).

Algorithms:

- **POS tagging (Part of Speech):** tags words as noun, verb, etc.

- “Supplier ships goods” → Supplier (NOUN), ships (VERB), goods (NOUN).
- **Noun phrase chunking:** extracts multi-word concepts.
 - Examples: “purchase order”, “shipping delay”.
- **Frequency-based filtering:** rank by how often they appear.
- **C-value / NC-value:** a scoring method for multi-word terms.
 - Example: “customer relationship management” scores higher than just “customer.”

Step 3. Taxonomy Induction (Building “is-a” hierarchies)

What: Organize concepts into parent-child trees.

Algorithms:

- **Hearst Patterns:** patterns in text that signal hierarchies.
 - “ERP systems such as SAP and Oracle” → Oracle is-a ERP system.
- **Clustering (distributional methods):** group terms that occur in similar contexts.
 - If “invoice” and “bill” both appear with “pay” and “send,” they may belong under “financial document.”

👉 Example Taxonomy:

```

Organization
├── Customer
└── Supplier
Document
├── Invoice
└── Contract
  
```

Step 4. Relation Extraction (non-hierarchy)

What: Find verbs and other relations between entities.

Algorithms:

- **Pattern Matching:**
 - “X supplies Y” → Supplier supplies Product.
 - “Invoice paid by Customer” → Invoice paidBy Customer.
- **OpenIE (Open Information Extraction):** pulls triples (Subject, Relation, Object) directly.
 - “Acme ships goods to Vienna” → (Acme, ships, Vienna).
- **NER + Relation models (neural):** detects entities + relations jointly.

Step 5. Entity Recognition (NER)

What: Detect mentions of specific entities (names, places, products).

Algorithms:

- **Rule-based (regex, dictionaries):**
 - Detects invoice numbers (INV-1234).
- **ML-based:**
 - CRF, BiLSTM, or Transformers (BERT).

Example:

- “Siemens shipped goods to Vienna.”
 - Siemens → ORGANIZATION
 - Vienna → LOCATION
 - goods → PRODUCT

Step 6. Entity Linking / Disambiguation

What: Link detected entities to ontology nodes.

Algorithms:

- **String similarity:** compare names (Levenshtein, Jaccard).
 - “IBM Corp” ≈ “International Business Machines.”
- **Embedding similarity:** represent text as vectors, find nearest neighbors.
- **Graph context:** if both names are linked to same VAT number, they’re the same.

Step 7. Ontology Rules & Constraints

What: Add validation rules to make ontology useful.

Languages/Tools:

- **OWL (Web Ontology Language)** → for logic-based ontologies.
- **SHACL (Shapes Constraint Language)** → for validation rules.

👉 Examples:

- Invoice must have exactly 1 Supplier.
- Shipment must have a delivery date.

Step 8. Evaluation & Refinement

What: Measure and improve quality.

Metrics:

- **Coverage:** % of data terms mapped to ontology.
 - **Precision:** % of extracted relations that are correct.
 - **Human validation:** business experts review suggestions (“Yes/No” UX).
-

3. Key Algorithms Explained (with Analogies)

- **Hearst Patterns:** pre-defined text templates for “is-a” relations.
 - Analogy: like Mad Libs sentences that always reveal a family relation.
 - **Word Embeddings:** convert words into math vectors so similar words are “closer.”
 - Analogy: a map where “invoice” and “bill” are next to each other.
 - **OpenIE:** pulls fact triples from text automatically.
 - Analogy: a sentence is a Lego block → algorithm breaks it into 3 Lego pieces: who, does what, to whom.
 - **Levenshtein Distance:** measures how many edits it takes to turn one string into another.
 - “kitten” → “sitting” = 3 edits.
 - **OWL/SHACL:** languages to describe rules/constraints in your ontology.
 - Analogy: building codes for your data house: “every invoice must have a supplier.”
-

4. Example Walkthrough

Sentence:

- “Siemens issued invoice INV-1234 for a shipment to Vienna on 12 July 2023.”

NER:

- Siemens → ORG
- INV-1234 → INVOICE
- Vienna → LOCATION
- 12 July 2023 → DATE

Relation Extraction:

- (Siemens, issued, Invoice)
- (Invoice, forShipment, Shipment)
- (Shipment, to, Vienna)

Linking:

- Siemens → Supplier node in ontology.
- INV-1234 → Invoice entity.

Ontology Rule Check (SHACL):

- Invoice has Supplier →  Valid.

👉 Output in graph form:

```
(Supplier: Siemens) --issued--> (Invoice: INV-1234) --forShipment-->
(Shipment) --to--> (Location: Vienna)
```

Recap

To build ontology extraction:

- Collect terms.
- Extract candidate concepts.
- Build hierarchies (is-a).
- Extract other relations.
- Detect entities in text.
- Link them to ontology.
- Add rules/constraints.
- Evaluate and refine.

You now know:

- The terms (NER, embeddings, Hearst patterns, OpenIE, etc.).
 - The definitions (what they are).
 - The examples (how they work in practice).
-

Additional Topics for Production-Grade Ontology Extraction

Ontology Alignment and Merging

- **Use case:** Integrating multiple schemas or ontologies (e.g., ERP + CRM).
- **Tools:** LogMap, AML (AgreementMakerLight).
- **Challenges:** conflicting definitions, duplicate concepts.

Knowledge Graph (KG) Construction

- Ontology defines schema; KG holds real-world data.
- Tools: RDFLib, Neo4j, GraphDB.
- Tip: Populate KG using extracted triples + validate with SHACL.

Semantic Embeddings for Concepts

- Use **OWL2Vec**, **KG-BERT**, or **RDF2Vec** for semantic similarity.
- Enables fuzzy matching, smarter linking, and better search.

Pipeline Orchestration

- Turn your extraction logic into a reusable pipeline.
- Tools: Apache NiFi, Airflow, Dagster.
- From CSVs/contracts → ontology → KG → SHACL validation.

Human-in-the-Loop Review

- Let users approve/reject suggested triples, concepts.
- Tools: Streamlit, Gradio, Jupyter widgets.
- Benefit: better trust + quality.

Active Learning Feedback Loop

- Improve ML-based extraction models with user feedback.
- Store user-approved relations, fine-tune models over time.

Ontology Quality Metrics

- **Beyond coverage & precision:**
 - Coherence (no contradictions).
 - Conciseness (no redundancy).
 - Completeness (all key domain concepts included).

External Ontology Integration

- Reuse standard ontologies:
 - `schema.org`, FOAF, FIBO (finance), GoodRelations.
- Use `rdfs:subClassOf`, `owl:sameAs` to align concepts.
- Benefit: Interoperability and reduced effort.

Study Glossary – Ontology & Semantics in Knowledge Graphs

Ontology

A classification scheme that describes categories *and* multiple kinds of relationships (not just hierarchies).

Example: An ontology can say “headphones are *PART_OF* personal audio equipment and *COMPATIBLE_WITH* smartphones.”

Semantics

The meaning encoded into a knowledge graph through an organizing principle. Makes data *self-describing*.

Example: Instead of raw text “Rosa → Berlin,” semantics adds “Rosa *LIVES_IN* Berlin since 2020.”

Taxonomy

A hierarchical classification (broader → narrower). A simpler organizing principle than an ontology.

Example: Fruit → Citrus → Orange.

Organizing Principle

The structure that makes a graph into a knowledge graph (taxonomy, ontology, schema, etc.).

Example: A product catalog organized by categories and relationships.

Semantic Bridge

An ontology linking the *same concept* across systems/domains to enable interoperability.

Example: Mapping “Car” in one system to “Automobile” in another.

Just-Enough Semantics

Design principle: add only as much meaning as current use cases need. Avoid overcomplicated ontologies.

Example: Start with “Customer BUYS Product” before modeling detailed purchase intents.

OWL (Web Ontology Language)

A W3C standard for defining expressive ontologies (complex classes, restrictions, inferencing).

Example: Define that “*All mammals are warm-blooded animals.*”

RDFS (RDF Schema)

A simpler standard for defining classes and relationships in RDF.

Example: Define that “*Person is a class, Employee is a subclass of Person.*”

SKOS (Simple Knowledge Organization System)

A W3C standard for controlled vocabularies and taxonomies.

Example: Use SKOS to model a library subject hierarchy.

Entity Disambiguation / Named-Entity Linking

Process of matching text mentions to ontology entities.

Example: Linking “UK” to the ontology entry *United Kingdom (country)*.

Semantic Search

Search that uses ontology meaning instead of keywords (“things, not strings”).

Example: Querying for “mobile phone” returns iPhone and Samsung Galaxy.

Reasoning / Inferencing

Automatic derivation of new facts from ontology rules.

Example: If “*all birds can fly*” and “*penguin is a bird*”, reasoning tells us “*penguin can fly*” (or corrects it if exceptions exist).

Layered Ontologies

Multiple modular ontologies stacked to bridge domains.

Example: One ontology for products, another for promotions, combined to answer “*What discounts apply to wireless headphones?*”

Ontology as a Contract

Ensures consistent meaning across systems or organizations.

Example: In finance, FIBO ontology standardizes terms like “*bond*,” “*equity*,” and “*derivative*.”

Standards vs. Custom Ontology

Decision to adopt an existing standard ontology or create one.

Example: Use **Schema.org** for web content, or build a custom ontology for your company’s product catalog.

Types of Semantics in Knowledge Graphs

1. Structural Semantics

Meaning derived from the *graph structure itself* (nodes, edges, properties).

- Relationships carry minimal semantics (e.g., “*Fred LIVES_IN London*”).
 - Still useful for queries and traversals.
 - **Closest to property graphs.**
-

2. Taxonomical Semantics

Meaning via *hierarchical relationships* (broader → narrower).

- Lets you reason about categories and subclasses.
 - **Example:** “*Headphones SUBCATEGORY_OF Audio Equipment*”.
 - Supports reasoning like “*If X is a headphone, then X is audio equipment.*”
-

3. Ontological Semantics

Richer semantics using many relationship types, not just hierarchy.

- Defines domain concepts + complex associations (PART_OF, COMPATIBLE_WITH, CAUSES, etc.).
 - Can encode constraints (transitive, symmetric, inverse).
 - **Example:** “*iPhone 12 UPSELL_TO iPhone 12 Pro*”.
 - Enables *horizontal* reasoning across categories.
-

4. Formal Semantics (Standards-based)

Expressing meaning in **machine-interpretable languages** like RDF Schema, OWL, or SKOS.

- Supports automated reasoning and inferencing.
 - **Example:** An OWL reasoner infers that “*If A SUBCLASS_OF B and B SUBCLASS_OF C, then A SUBCLASS_OF C.*”
-

5. Pragmatic / Just-Enough Semantics

Semantics aligned with the *actual use case* — don't over-model.

- Flexible and incremental.
 - **Example:** Only model “*Customer BUYS Product*” at first, add “*Customer REVIEWS Product*” later when needed.
-

6. Cross-Domain Semantics (Semantic Bridges)

Semantics used for interoperability across systems/domains.

- Ontologies align vocabularies so systems can talk to each other.
 - **Example:** Mapping “*Car*” in System A to “*Automobile*” in System B.
-

7. Lexical / Linguistic Semantics (Natural Language)

Used when connecting graphs with language resources (WordNet, embeddings).

- Supports natural language querying & similarity.
- **Example:** Recognizing that “*couch*” $\approx$ “*sofa*” in semantic search.